

**LAPORAN PRAKTIKUM**  
**PEMROGRAMAN BERORIENTASI OBJEK**  
**MODUL 03 – Berorientasi Objek**



Maysa Azhra Fauziyyah

103132430005

S1SD-05-01

**PROGRAM STUDI SAINS DATA**  
**FAKULTAS INFORMATIKA**  
**TELKOM UNIVERSITY PURWOKERTO**  
**2026**

## A. Dasar Teori

### 3.1 Pendahuluan

Pemrograman berorientasi objek adalah metode membangun program dengan cara membungkus data dan fungsi jadi satu kesatuan yang disebut objek, dan antar objek ini bisa saling berinteraksi atau mewarisi sifat satu sama lain. Ada 5 konsep dasar yang perlu dipahami:

- Abstraksi, yaitu cuma ngambil hal-hal penting dari suatu objek aja (atribut dan perilakunya), yang nggak penting diabaikan.
- Enkapsulasi, yaitu data dan method dibungkus jadi satu dalam kelas, biasanya data-nya disembunyikan (private) dan cuma bisa diakses lewat method yang public. Tujuannya biar data nggak bisa diutak-atik sembarangan dari luar.
- Pewarisan, bikin kelas baru yang mewarisi sifat dari kelas yang sudah ada, terus bisa ditambah karakteristik baru lagi.
- Polimorfisme, satu method/interface yang sama tapi hasilnya bisa beda-beda tergantung objeknya.
- Message, cara objek saling komunikasi, biasanya lewat pemanggilan method.

### 3.2 Kelas

Kelas itu semacam blueprint/cetakan buat bikin objek. Dari satu kelas bisa dibikin banyak objek. Di dalam kelas ada atribut (data) dan method (fungsi).

### 3.3 Object

Objek adalah hasil dari kelas yang sudah di-instansiasi pakai new. Jadi kelas itu rancangannya, objek itu barang jadinya.

### 3.4 Field

Field itu atribut/variabel yang ada di dalam kelas. Ada 2 jenis:

- Class field – pakai static, nilainya sama dan dipakai oleh semua objek, diakses lewat nama kelasnya langsung.
- Instance field – nggak pakai static, tiap objek punya nilai sendiri-sendiri yang beda.

### 3.5 Method

Method itu fungsi di dalam kelas. Kalau ngembaliin nilai disebut function (ada return), kalau nggak ngembaliin nilai disebut procedure (void). Cara ngirim parameter ke method ada 2:

- Pass by value – buat tipe data primitif, yang dikirim cuma salinan nilainya, jadi kalau diubah di dalam method, nilai aslinya nggak ikut berubah.
- Pass by reference – buat array/objek, yang dikirim itu alamat memorinya, jadi kalau diubah di dalam method, nilai aslinya ikut berubah juga.

### 3.6 Keyword “this”

this dipakai buat nunjuk ke objek yang lagi aktif. Biasanya dipakai pas nama parameter sama dengan nama field, supaya nggak ketuker. Selain itu this(...) juga bisa dipakai buat manggil constructor lain dari constructor yang sama.

## B. Guided

### 1. Abstraksi-Kucing

```
#Kucing
package com.maysa.projectmodul03.Guided.Abstraksi;

public class Kucing {
    String nama;
    String ras;

    void bersuara(){
        System.out.println(nama + ras + "meow");
    }
}

#Abstraksi
Kucing
package com.maysa.projectmodul03.Guided.Abstraksi;

public class Abstraksi {
    public static void main(String[] args){
        Kucing aren = new Kucing ();

        aren.nama = "Aren Gentong";
        aren.ras = "Mujaer";
        aren.bersuara ();

    }
}
```

### Output :

```
-----< com.Maysa:ProjectModul03 >-----
Building ProjectModul03 1.0-SNAPSHOT
from pom.xml
-----[ jar ]-----

--- resources:3.4.0:resources (default-resources) @ ProjectModul03 ---
skip non existing resourceDirectory C:\Users\ASUS\Documents\NetBeansProjects\ProjectModul03\src\main\resources

--- compiler:3.15.0:compile (default-compile) @ ProjectModul03 ---
Nothing to compile - all classes are up to date.

--- exec:3.6.3:exec (default-cli) @ ProjectModul03 ---
Aren GentongMujaermeow

BUILD SUCCESS

Total time: 1.467 s
Finished at: 2026-10-02T14:45:22+07:00
-----
```

### Kesimpulan:

kelas Kucing cuma nyimpen 2 atribut (nama, ras) dan 1 method (bersuara()), ini contoh abstraksi cuma ngambil hal penting dari objek kucing aja. Outputnya nyambung tanpa spasi karena di println-nya memang nggak dikasih spasi pemisah antar variabel.

## 2. Enkapulasi - Rekening

```
#Rekening
package com.maysa.projectmodul03.Guided.Enkapsulasi;

public class Rekening {
    private int saldo = 0;

    public void tambahSaldo(int jumlah){
        saldo = saldo + jumlah;
        System.out.println("Saldo berhasil ditambahkan");
    }

    public void tampilkanSaldo(){
        System.out.println("Saldo Anda: " + saldo);
    }
}

#Main-Enkapulasi
package com.maysa.projectmodul03.Guided.Enkapsulasi;

public class Enkapsulasi {

    public static void main(String[] args) {
        Rekening rek = new Rekening ();

        rek.tambahSaldo(20000000);
        rek.tampilkanSaldo();
    }
}
```

### Output :

```
-----< com.Maysa:ProjectModul03 >-----
Building ProjectModul03 1.0-SNAPSHOT
  from pom.xml
-----[ jar ]-----

--- resources:3.4.0:resources (default-resources) @ ProjectModul03 ---
skip non existing resourceDirectory C:\Users\ASUS\Documents\NetBeansProjects\ProjectModul03\src\main\resources

--- compiler:3.15.0:compile (default-compile) @ ProjectModul03 ---
Nothing to compile - all classes are up to date.

--- exec:3.6.3:exec (default-cli) @ ProjectModul03 ---
Saldo berhasil ditambahkan
Saldo Anda: 20000000
-----
BUILD SUCCESS
-----
Total time:  2.149 s
Finished at: 2026-10-02T15:18:24+07:00
-----
```

Kesimpulan:

field saldo dibuat private jadi nggak bisa diubah langsung dari luar, harus lewat method public tambahSaldo(). Ini contoh penyembunyian data di enkapsulasi.

### 3. 3.4 Field – Circle01

```
#Circle01
package com.maysa.projectmodul03.Guided.Enkapsulasi;

public class Circle01 {
    public static final double PI = 3.14159;

    public static double radiansToDegrees(double rads){
        return rads * 180 / PI;
    }

    public double r;

    public double area() {
        return PI * r * r;
    }

    public double circumference() {
        return 2 * PI * r;
    }
}

#MainCircle
package com.maysa.projectmodul03.Guided.Enkapsulasi;

public class MainCircle01 {
    public static void main(String[] args) {

        Circle01 lingkaran = new Circle01();

        lingkaran.r = 7;

        System.out.println("Jari-jari : " + lingkaran.r);
        System.out.println("Luas      : " + lingkaran.area());
        System.out.println("Keliling : " + lingkaran.circumference());

        double radian = 3.14159;
        System.out.println("Radian   : " + radian);
        System.out.println("Derajat  : " + Circle01.radiansToDegrees(radian));
    }
}
```

## Output :

```
-----< com.Maysa:ProjectModul03 >-----
Building ProjectModul03 1.0-SNAPSHOT
  from pom.xml
-----[ jar ]-----

--- resources:3.4.0:resources (default-resources) @ ProjectModul03 ---
skip non existing resourceDirectory C:\Users\ASUS\Documents\NetBeansProjects\ProjectModul03\src\main\resourc

--- compiler:3.15.0:compile (default-compile) @ ProjectModul03 ---
Nothing to compile - all classes are up to date.

--- exec:3.6.3:exec (default-cli) @ ProjectModul03 ---
Jari-jari : 7.0
Luas      : 153.93791
Keliling  : 43.98226
Radian    : 3.14159
Derajat   : 180.0

BUILD SUCCESS

Total time: 1.946 s
Finished at: 2026-10-02T15:21:30+07:00
-----
|
```

## Kesimpulan:

PI itu class field (static final) jadi bisa dipanggil langsung lewat Circle01.radiansToDegrees() tanpa bikin objek dulu. Sedangkan r itu instance field, nilainya cuma punya objek lingkaran itu sendiri.

## 4. 3.5 Enkapsulasi – Method

```
package com.maysa.projectmodul03.Guided.Enkapsulasi;

public class Method {
    int i, j;

    Method(int a, int b) {
        i = a;
        j = b;
    }

    // passed by value dengan parameter tipe data primitif
    void calculate(int m, int n) {
        m = m * 10;
        n = n / 2;
    }

    // passed by reference dengan parameter berupa tipe data class
    void calculate(Method e) {
        e.i = e.i * 10;
        e.j = e.j / 2;
    }
}
```

```

#MainMethod
package com.maysa.projectmodul03.Guided.Enkapsulasi;

public class MainMethod {

    public static void main(String[] args) {
        int x, y;
        Method z;
        z = new Method(50, 100);
        x = 10;
        y = 20;
        System.out.println("Nilai sebelum passed by value : ");
        System.out.println("x = " + x);
        System.out.println("y = " + y);
        //passed by value
        z.calculate(x, y);
        System.out.println("Nilai sesudah passed by value : ");
        System.out.println("x = " + x);
        System.out.println("y = " + y);
        System.out.println("Nilai sebelum passed by reference : ");
        System.out.println("z.i = " + z.i);
        System.out.println("z.j = " + z.j);
        //passed by reference
        z.calculate(z);
        System.out.println("Nilai sesudah passed by reference : ");
        System.out.println("z.i = " + z.i);
        System.out.println("z.j = " + z.j);
    }
}

```

## Output:

```

-----< com.Maysa:ProjectModul03 >-----
Building ProjectModul03 1.0-SNAPSHOT
from pom.xml
-----[ jar ]-----

--- resources:3.4.0:resources (default-resources) @ ProjectModul03 ---
skip non existing resourceDirectory C:\Users\ASUS\Documents\NetBeansProjects\ProjectModul03\src\main\resources

--- compiler:3.15.0:compile (default-compile) @ ProjectModul03 ---
Recompiling the module because of changed source code.
Compiling 14 source files with javac [debug release 26] to target\classes

--- exec:3.6.3:exec (default-cli) @ ProjectModul03 ---
Nilai sebelum passed by value :
x = 10
y = 20
Nilai sesudah passed by value :
x = 10
y = 20
Nilai sebelum passed by reference :
z.i = 50
z.j = 100
Nilai sesudah passed by reference :
z.i = 500
z.j = 50
-----
BUILD SUCCESS
-----
Total time: 1.536 s
Finished at: 2026-10-02T15:26:13+07:00
-----

```

Kesimpulan:

x dan y nilainya tetap walau udah dikirim ke calculate(int, int), karena itu tipe primitif jadi cuma salinannya yang dikirim (pass by value). Beda sama z.i dan z.j yang berubah setelah calculate(z) dipanggil, karena z itu objek jadi yang dikirim alamat memorinya (pass by reference), makanya perubahan di dalam method dibawa sampai ke luar.

### 5. 3.6 Enkapsulasi – Keyword

```
#Keyword
package com.maysa.projectmodul03.Guided.Enkapsulasi;

public class Keyword {

    private String pencipta;
    private String judul;

    public void IsiParam(String judul, String pencipta) {
        this.judul = judul;
        this.pencipta = pencipta;
    }

    public void cetakKeLayar() {
        if (judul == null && pencipta == null) {
            return;
        }
        System.out.println("Judul : " + judul + ", pencipta : "
            + pencipta);
    }
}

#MainKeyword
package com.maysa.projectmodul03.Guided.Enkapsulasi;

public class MainKeyword {

    public static void main(String[] args) {
        Keyword a = new Keyword();
        a.IsiParam("God Will Make A Way", "Don Moen ");
        a.cetakKeLayar();
    }
}
```

## Output

```
-----< com.Maysa:ProjectModul03 >-----
] Building ProjectModul03 1.0-SNAPSHOT
  from pom.xml
-----[ jar ]-----

] --- resources:3.4.0:resources (default-resources) @ ProjectModul03 ---
- skip non existing resourceDirectory C:\Users\ASUS\Documents\NetBeansProjects\ProjectModul03\src\main\resources

] --- compiler:3.15.0:compile (default-compile) @ ProjectModul03 ---
  Recompiling the module because of changed source code.
- Compiling 14 source files with javac [debug release 26] to target\classes

] --- exec:3.6.3:exec (default-cli) @ ProjectModul03 ---
- Judul : God Will Make A Way, pencipta : Don Moen
-----
BUILD SUCCESS
-----
Total time:  1.873 s
Finished at: 2026-10-02T15:31:45+07:00
-----
.
```

## Kesimpulan:

nama parameter di IsiParam(String judul, String pencipta) sama persis dengan nama field di kelasnya, makanya butuh this.judul dan this.pencipta biar Java tau itu yang dimaksud field punya objek, bukan parameter-nya.

## 6. 3.6 Enkapsulasi – KeywordBuku

```
package com.maysa.projectmodul03.Guided.Enkapsulasi;

public class KeywordBuku {

    private String pengarang;
    private String judul;

    private KeywordBuku() {
        this("Rumah Kita", "GoodBles");
    }
    // this ini digunakan untuk memanggil konstruktor yang
    // menerima dua parameter
    private KeywordBuku(String judul, String pengarang) {
        this.judul = judul;
        this.pengarang = pengarang;
    }

    private void cetakKeLayar() {
        System.out.println("Judul : " + judul + " Pengarang : "
            + pengarang);
    }

    public static void main(String[] args) {
        KeywordBuku a, b;
        a = new KeywordBuku("Jurassic Park", "Michael Chricton");
        b = new KeywordBuku();
        a.cetakKeLayar();
        b.cetakKeLayar();
    }
}
```

## Output

```
-----< com.Maysa:ProjectModul03 >-----
| Building ProjectModul03 1.0-SNAPSHOT
|   from pom.xml
|-----[ jar ]-----

| --- resources:3.4.0:resources (default-resources) @ ProjectModul03 ---
| skip non existing resourceDirectory C:\Users\ASUS\Documents\NetBeansProjects\ProjectModul03\src\main\resources

| --- compiler:3.15.0:compile (default-compile) @ ProjectModul03 ---
| Recompiling the module because of changed source code.
| Compiling 14 source files with javac [debug release 26] to target\classes

| --- exec:3.6.3:exec (default-cli) @ ProjectModul03 ---
| Judul : Jurassic Park Pengarang : Michael Chricton
| Judul : Rumah Kita Pengarang : GoodBles
|-----
| BUILD SUCCESS
|-----
| Total time: 1.558 s
| Finished at: 2026-10-02T15:43:11+07:00
|-----
```

## Kesimpulan:

constructor kosong KeywordBuku() manggil this("Rumah Kita", "GoodBles"), jadi dia manggil constructor lain yang udah punya parameter, nggak perlu nulis ulang kode buat ngisi field-nya. Constructor-nya juga private jadi objeknya cuma bisa dibuat dari dalam kelas itu sendiri.

## C. Unguided

### 1. Soal Unguided 1

# Soal Unguided — Modul 3: Pemrograman Berorientasi Objek

### Penerapan OOP Sederhana untuk Pengolahan Data

---

## Petunjuk Umum

Buatlah program Java berdasarkan cerita berikut. Rancang sendiri struktur program yang diperlukan sebelum menuliskan kode.

- Tentukan sendiri field, constructor, dan method yang diperlukan, mengikuti hint yang disediakan.
- Data di dalam object tidak boleh diakses atau diubah langsung dari luar class-nya.
- Gunakan nama parameter yang sama dengan nama field pada constructor (sehingga `this` perlu dipakai secara eksplisit).
- Program ini harus memuat dua jenis field: **\*\*instance field\*\*** (milik masing-masing object) dan **\*\*class field\*\*** (milik class, dipakai bersama).
- Tidak perlu inheritance, overriding, maupun abstract class.

---

## Soal — Pembersihan dan Analisis Data Suhu Mingguan

Sebuah tim mengumpulkan suhu harian suatu kota selama satu minggu (7 hari) dari sensor otomatis. Data ini sudah tersedia sebagai array di `main` sebelum diserahkan untuk diolah.

Pada salah satu hari, sensor gagal mengirim data. Hari yang gagal terekam ditandai dengan sebuah nilai penanda pada array, karena suhu asli tidak mungkin sebesar itu. Nilai penanda ini sama untuk semua data suhu yang diolah kapan pun dan oleh siapa pun — bukan sesuatu yang berbeda-beda tiap kali data dibuat — sehingga sebaiknya ditulis satu kali saja sebagai aturan tetap milik class, bukan diketik ulang sebagai angka `-1.0` di setiap method yang membutuhkannya.

Buatlah satu class yang bertugas mengolah data suhu ini — memeriksa apakah ada data kosong, mengisinya, dan menghasilkan statistik dasar dari data yang sudah bersih.

### Kebutuhan program

Class ini harus dapat:

1. **\*\*Menyimpan data suhu 7 hari\*\***, dibuat dari array suhu mentah yang sudah ada di `main` (index array mewakili hari).
2. **\*\*Menyimpan nilai penanda data kosong (`-1.0`)\*\*** sebagai aturan tetap milik class\*\*, yaitu class field yang nilainya tidak dapat diubah dan dipakai bersama oleh semua object.
3. **\*\*Menampilkan seluruh data apa adanya\*\***, menandai hari yang kosong secara jelas.
4. **\*\*Mencari apakah ada hari yang kosong, dan di index berapa\*\***, karena index inilah yang nanti dipakai untuk proses pengisian data.
5. **\*\*Mengisi hari yang kosong\*\***, menggunakan index dari poin 4, dengan nilai perkiraan berdasarkan rata-rata suhu satu hari sebelum dan satu hari sesudahnya.
6. **\*\*Menghitung rata-rata suhu\*\*** dari data yang sudah bersih.

### Hint

> Hint di bawah ini adalah acuan minimal. Anda tetap boleh menambah method atau menyesuaikan detail selama seluruh kebutuhan di atas terpenuhi.

**\*\*Nama class:\*\*** `PengolahSuhu`

**\*\*Field yang disarankan:\*\***

| Field          | Jenis          | Tipe       | Keterangan                                                                    |
|----------------|----------------|------------|-------------------------------------------------------------------------------|
| `suhuHarian`   | Instance field | `double[]` | Menyimpan array suhu 7 hari; berbeda untuk setiap object                      |
| `NILAI_KOSONG` | Class field    | `double`   | Nilai penanda data kosong (`-1.0`); sama untuk semua object dan tidak berubah |

**\*\*Method yang disarankan:\*\***

| Method              | Kebutuhan | Return                                                                             |
|---------------------|-----------|------------------------------------------------------------------------------------|
| `tampilkanData()`   | 3         | `void`                                                                             |
| `cariIndexKosong()` | 4         | `int` — index hari yang bernilai `NILAI_KOSONG`, atau `-1` apabila tidak ditemukan |

```
| `isiDataKosong()` | 5 | `void` — memanggil `cariIndexKosong()` di dalamnya; jika  
hasilnya bukan `-1`, isi index tersebut |  
| `hitungRataRata()` | 6 | `double` |
```

**\*\*Rumus imputasi (pengisian data kosong) untuk time series:\*\***

Untuk index hari `i` yang kosong:

```
...
```

```
suhuHarian[i] = ( suhuHarian[i-1] + suhuHarian[i+1] ) / 2
```

```
...
```

Ini adalah metode imputasi paling dasar untuk data deret waktu, yaitu **\*\*mengisi nilai yang hilang dengan rata-rata dari titik data tetangga terdekatnya\*\*** (sebelum dan sesudah), dengan asumsi nilai yang hilang berada di antara dua kondisi yang sudah diketahui dan tidak berubah drastis.

### Data suhu (array di `main`)

Gunakan array berikut sebagai data suhu 7 hari (Hari 1 = index 0):

```
```java
```

```
double[] suhuHarian = { 30.4, 24.3, 26.8, -1.0, 31.4, 30.8, 32.9 };
```

```
```
```

Hari yang kosong: **\*\*Hari 4\*\***.

### Alur program di `main`

1. Siapkan array `suhuHarian` seperti di atas.
2. Buat object `PengolahSuhu` dari array `suhuHarian` tersebut. Tampilkan data awal (`tampilkanData()`), lalu panggil `cariIndexKosong()` dan tampilkan index yang ditemukan.
3. Jalankan `isiDataKosong()`.
4. Tampilkan data akhir yang sudah bersih (`tampilkanData()`) beserta rata-rata suhunya.
5. Tampilkan kembali isi array `suhuHarian` di `main` (bukan lewat object `PengolahSuhu`, melainkan variabel array yang sudah disiapkan di langkah 1). Amati bahwa array ini pun sudah berubah, padahal tidak pernah diubah langsung di `main`, lalu jelaskan sebabnya dalam beberapa kalimat sebagai komentar pada kode program.

### Output yang diharapkan

```
...
```

```
=== Data Suhu Awal ===
```

```
Hari 1 : 30.4°C
```

```
Hari 2 : 24.3°C
```

```
Hari 3 : 26.8°C
```

```
Hari 4 : (kosong)
```

```
Hari 5 : 31.4°C
```

```
Hari 6 : 30.8°C
```

```
Hari 7 : 32.9°C
```

Index hari kosong (dimulai dari 0): 3

==== Data Suhu Setelah Pengisian ====

Hari 1 : 30.4°C

Hari 2 : 24.3°C

Hari 3 : 26.8°C

Hari 4 : 29.1°C

Hari 5 : 31.4°C

Hari 6 : 30.8°C

Hari 7 : 32.9°C

Rata-rata : 29.39°C

Isi array suhuHarian di main setelah isiDataKosong() dijalankan:

[30.4, 24.3, 26.8, 29.1, 31.4, 30.8, 32.9]

(ikut berubah: constructor menyimpan referensi array yang sama)

``

---

## Yang perlu diperhatikan saat mengerjakan

- Amati baik-baik dua baris terakhir pada output di atas. Array `suhuHarian` yang dicetak itu adalah variabel di `main`, bukan diambil lewat object `PengolahSuhu` — dan nilainya sudah berubah, padahal `main` tidak pernah memanggil operasi pengubahan itu sendiri. Ini terjadi karena array bertipe **reference**: constructor `PengolahSuhu` menyimpan array yang sama persis (bukan menyalinnya), sehingga perubahan yang dilakukan `isiDataKosong()` pada object otomatis terlihat juga dari sisi `main`. Jelaskan hal ini dalam beberapa kalimat sebagai komentar pada kode program.

- Nilai penanda `-1.0` harus ditulis satu kali saja sebagai class field, lalu dipakai oleh setiap method yang membutuhkannya. Jangan mengetik ulang angka `-1.0` di dalam method.

- Rumus imputasi mengasumsikan hari sebelum dan sesudah hari yang kosong tersedia sebagai data asli (bukan ikut kosong). Data pada soal ini sudah dijamin memenuhi asumsi tersebut.

Jawaban :

```
package com.maysa.projectmodul03.Unguided;

public class PengolahSuhu {

    private double[] suhuHarian; //kurung siku buat array, biasanya ada isinya dan isinya
    lebih dari satu//
    private static final double NILAI_KOSONG = -1.0;

    public PengolahSuhu(double[] suhuHarian) { //constructor:ciri'nya namanya sama
    dengan nama class
        this.suhuHarian = suhuHarian; //untuk membuat objek suhu harian
    }

    public void tampilkanData() { //procedure, tidak akan mengembalikan nilai
    //perulangan
        for (int i = 0; i < suhuHarian.length; i++) { //jika i<jumlah suhuharian, maka
    perulangan berhenti

            if (suhuHarian[i] == NILAI_KOSONG) {
                System.out.println("Hari " + (i + 1) + " : (kosong)");
            } else {
                System.out.println(
                    "Hari " + (i + 1) + " : " + suhuHarian[i] + "\u00B0C"
                );
            }
        }
    }

    public int cariIndexKosong() {
        for (int i = 0; i < suhuHarian.length; i++) {

            if (suhuHarian[i] == NILAI_KOSONG) {
                return i;
            }
        }

        return -1;
    }

    public void isiDataKosong() { //isinya index yang kosong, dengan memanggil index
    kosong
        int indexKosong = cariIndexKosong();

        if (indexKosong != -1) {

            suhuHarian[indexKosong] =
                (suhuHarian[indexKosong - 1]
                    + suhuHarian[indexKosong + 1]) / 2;
        }
    }
}
```

```

public double hitungRataRata() {
    double total = 0;

    for (double suhu : suhuHarian) {
        total += suhu;
    }

    return total / suhuHarian.length; //total data suhu harian
}
}

#Main
package com.maysa.projectmodul03.Unguided;

import java.io.PrintStream;
import java.io.UnsupportedEncodingException;
import java.util.Arrays;

public class Main {

    public static void main(String[] args) {

        // Paksa output Java jadi UTF-8 supaya simbol derajat terbaca
        try {
            System.setOut(new PrintStream(System.out, true, "UTF-8"));
        } catch (UnsupportedEncodingException e) {
            e.printStackTrace();
        }

        // Data suhu 7 hari
        double[] suhuHarian = {
            30.4, 24.3, 26.8, -1.0, 31.4, 30.8, 32.9
        };

        // Membuat object PengolahSuhu
        PengolahSuhu pengolah = new PengolahSuhu(suhuHarian);

        // Menampilkan data awal
        System.out.println("=== Data Suhu Awal ===");
        pengolah.tampilkanData();

        // Mencari index yang kosong
        int indexKosong = pengolah.cariIndexKosong();

        System.out.println();
        System.out.println(
            "Index hari kosong (dimulai dari 0): " + indexKosong
        );

        // Mengisi data yang kosong
        pengolah.isiDataKosong();
    }
}

```

```

// Menampilkan data setelah pengisian
System.out.println();
System.out.println("=== Data Suhu Setelah Pengisian ===");
pengolah.tampilkanData();

// Menghitung rata-rata
double rataRata = pengolah.hitungRataRata();

System.out.printf(
    "%nRata-rata : %.2f\u00B0C%n",
    rataRata
);

// Menampilkan array yang ada di main
System.out.println();
System.out.println(
    "Isi array suhuHarian di main setelah isiDataKosong() dijalankan:"
);
System.out.println(Arrays.toString(suhuHarian));
}
} package com.maysa.projectmodul03.Unguided;

import java.io.PrintStream;
import java.io.UnsupportedEncodingException;
import java.util.Arrays;

public class Main {

    public static void main(String[] args) {

        // Paksa output Java jadi UTF-8 supaya simbol derajat terbaca
        try {
            System.setOut(new PrintStream(System.out, true, "UTF-8"));
        } catch (UnsupportedEncodingException e) {
            e.printStackTrace();
        }

        // Data suhu 7 hari
        double[] suhuHarian = {
            30.4, 24.3, 26.8, -1.0, 31.4, 30.8, 32.9
        };

        // Membuat object PengolahSuhu
        PengolahSuhu pengolah = new PengolahSuhu(suhuHarian);

        // Menampilkan data awal
        System.out.println("=== Data Suhu Awal ===");
        pengolah.tampilkanData();

        // Mencari index yang kosong

```

```

int indexKosong = pengolah.cariIndexKosong();

System.out.println();
System.out.println(
    "Index hari kosong (dimulai dari 0): " + indexKosong
);

// Mengisi data yang kosong
pengolah.isiDataKosong();

// Menampilkan data setelah pengisian
System.out.println();
System.out.println("=== Data Suhu Setelah Pengisian ===");
pengolah.tampilkanData();

// Menghitung rata-rata
double rataRata = pengolah.hitungRataRata();

System.out.printf(
    "%nRata-rata : %.2f\u00B0C%n",
    rataRata
);

// Menampilkan array yang ada di main
System.out.println();
System.out.println(
    "Isi array suhuHarian di main setelah isiDataKosong() dijalankan:"
);
System.out.println(Arrays.toString(suhuHarian));

// Array suhuHarian ini ikut berubah isinya walau tidak pernah diubah
// langsung di main, karena waktu object PengolahSuhu dibuat, constructor
// menyimpan referensi/alamat array yang sama persis (bukan menyalin
// datanya). Jadi saat isiDataKosong() mengubah data lewat object,

// perubahan itu otomatis kebaca juga di array ini.
}
}

```

#### Penjelasan source code:

Di kelas PengolahSuhu, data suhu 7 hari disimpan di suhuHarian (instance field, beda-beda tiap objek) dan nilai penanda data kosong disimpan di NILAI\_KOSONG (class field, -1.0, sama buat semua objek). Pas bikin objek lewat constructor, array yang dikirim itu disimpan referensinya aja, bukan disalin. Method tampilkanData() nge-print semua data pakai perulangan, kalau ketemu NILAI\_KOSONG ditandai "(kosong)". cariIndexKosong() nyari index yang nilainya NILAI\_KOSONG. isiDataKosong() manggil cariIndexKosong() dulu, baru ngisi index itu pakai rata-rata dari tetangga kiri-kanannya. hitungRataRata() tinggal jumlahin semua terus dibagi 7.

## Output :

```
-----< com.Maysa:ProjectModul03 >-----
| Building ProjectModul03 1.0-SNAPSHOT
|   from pom.xml
|-----[ jar ]-----

| --- resources:3.4.0:resources (default-resources) @ ProjectModul03 ---
| · skip non existing resourceDirectory C:\Users\ASUS\Documents\NetBeansProjects\ProjectModul03\src\main\resources

| --- compiler:3.15.0:compile (default-compile) @ ProjectModul03 ---
| · Recompiling the module because of changed source code.
| · Compiling 14 source files with javac [debug release 26] to target\classes

| --- exec:3.6.3:exec (default-cli) @ ProjectModul03 ---
| === Data Suhu Awal ===
| Hari 1 : 30.4°C
| Hari 2 : 24.3°C
| Hari 3 : 26.8°C
| Hari 4 : (kosong)
| Hari 5 : 31.4°C
| Hari 6 : 30.8°C
| Hari 7 : 32.9°C
|
| Index hari kosong (dimulai dari 0): 3
|
| === Data Suhu Setelah Pengisian ===
| Hari 1 : 30.4°C
| Hari 2 : 24.3°C
| Hari 3 : 26.8°C
| Hari 4 : 29.1°C
| Hari 5 : 31.4°C
| Hari 6 : 30.8°C
| Hari 7 : 32.9°C
|
| Rata-rata : 29.39°C
|
| Isi array suhuHarian di main setelah isiDataKosong() dijalankan:
| · [30.4, 24.3, 26.8, 29.1, 31.4, 30.8, 32.9]
|-----
| BUILD SUCCESS
|-----
| Total time: 3.445 s
| Finished at: 2026-10-05T00:59:06+07:00
|-----
```

### Penjelasan Output:

Hari 4 (index 3) ketemu kosong, terus diisi pakai rata-rata Hari 3 dan Hari 5:  $(26.8 + 31.4) / 2 = 29.1^{\circ}\text{C}$ . Rata-rata semua data setelah diisi:  $(30.4 + 24.3 + 26.8 + 29.1 + 31.4 + 30.8 + 32.9) / 7 = 29.3857\dots$ , dibulatin jadi  $29.39^{\circ}\text{C}$ . Yang menarik di baris terakhir, array suhuHarian yang ada di main ikut berubah juga padahal main nggak pernah ngubah-ngubah langsung. Ini kejadian karena pas bikin objek PengolahSuhu, yang disimpan itu alamat array-nya (referensi), bukan salinan datanya, jadi pas isiDataKosong() ngubah data di dalam objek, otomatis array yang di main juga ikut berubah.

## D. Kesimpulan

Dari bagian guided, jadi lebih paham gimana cara kerja abstraksi dan enkapsulasi (nyembunyiin data pakai private/public), bedanya class field sama instance field, dua cara ngirim parameter (pass by value buat primitif, pass by reference buat objek/array), sama kegunaan this buat nunjuk objek yang aktif atau manggil constructor lain. Di bagian unguided, semua konsep itu dipraktikin bareng dalam satu program pengolahan data suhu class field buat nilai tetap, instance field yang sifatnya reference sehingga perubahan di objek ikut dibawa ke main, sama beberapa method yang saling manggil buat ngecek dan ngisi data kosong. Intinya, paham konsep enkapsulasi dan sifat reference pada objek/array ini penting banget biar program yang dibikin lebih rapi dan nggak error pas datanya diproses.